Wcb

Programmierhandbuch (deutsch)

2026-10-26

Inhalt

index **3**
wcb **3**
wcbRef **16**
Lizenz **27**

index

Das Widget-Rückruffunktions-Paket Wcb 4.2

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

Wcb-Programmierhandbuch

Wcb-Befehlsreferenz

Stichwörter: index

wcb

Wcb-Programmierhandbuch

Für Wcb Version 4.2

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

Überblick
- Einige häufige Probleme
- Das Paket Wcb
- Woher bekomme ich es?
- Wie installiere ich es?
- Wie verwende ich es?

Beispiele
- Einige before-insert-Rückruffunktionen für entry-Widgets
- Eine selset-Rückruffunktion für ein listbox-Widget
- Eine activate-Rückruffunktion für ein listbox-Widget
- Sieben Rückruffunktionen für ein text-Widget

Überblick

Einige häufige Probleme

Viele, die mit Tcl/Tk programmieren, stehen vor Fragen wie diesen:

- Wie schränkt man die Menge der Zeichen ein, die der Benutzer in ein Tk- oder Ttk-entry, ein

BWidget Entry, eine Tk- oder Ttk-spinbox, eine Ttk-combobox, ein text- oder ein ctext-Widget tippen oder einfügen kann?
- Wie bearbeitet man die eingegebenen Zeichen, bevor sie in eines dieser Widgets eingefügt werden? Bei einem text- oder ctext-Widget: Wie ändert man Schrift, Farben oder andere Attribute der eingegebenen Zeichen?
- Wie setzt man eine Obergrenze für die Anzahl der Zeichen, die in ein Tk- oder Ttk-entry, ein BWidget Entry, eine Tk- oder Ttk-spinbox oder eine Ttk-combobox getippt oder eingefügt werden können?
- Wie schützt man Teile des Textes in einem Tk- oder Ttk-entry, einem BWidget Entry, einer Tk- oder Ttk-spinbox, einer Ttk-combobox, einem text- oder einem ctext-Widget davor, vom Benutzer geändert zu werden?
- Wie legt man Benachrichtigungen fest, die automatisch ausgelöst werden, nachdem Text in eines dieser Widgets eingefügt oder daraus gelöscht wurde?
- Wie legt man Aktionen fest, die automatisch aufgerufen werden, sobald die Einfügemarke in einem Tk- oder Ttk-entry, einem BWidget Entry, einer Tk- oder Ttk-spinbox, einer Ttk-combobox, einem text- oder einem ctext-Widget verschoben wird?
- Wie legt man einen Befehl fest, der automatisch aufgerufen wird, wenn ein listbox-Element, eine Tablelist-Zeile oder -Zelle, ein Ttk-treeview-Element oder ein Zeichenbereich in einem text- oder ctext-Widget ausgewählt wird?
- Wie schützt man beliebige oder alle Einträge eines listbox-, tablelist- oder Ttk-treeview-Widgets oder einen Zeichenbereich in einem text- oder ctext-Widget davor, ausgewählt zu werden?

In den meisten Büchern, FAQs, Newsgruppen-Beiträgen und Widget-Sammlungen finden Sie *einzelne* Lösungen für einige dieser Probleme über Widget-Bindungen. Dieser Weg fügt neue Bindungs-Tags hinzu oder ändert vorhandene, und er erweist sich recht oft als unvollständig.

Der Tk-Kern behandelt nur wenige dieser Probleme: In Tk 8.1 kam das virtuelle Ereignis `<<ListboxSelect>>` für listbox-Widgets hinzu, Tk ab Version 8.3 unterstützt Widget-Optionen zur Prüfung von entry-Widgets, und das spinbox-Widget (seit Tk 8.4) bietet dieselbe Prüfmöglichkeit. Tk ab Version 8.4 unterstützt außerdem den Zustand `disabled` für listbox-Widgets sowie das Kennzeichen `modified`, die virtuellen Ereignisse `<<Modified>>` und `<<Selection>>` und einen Rückgängig-/Wiederholen-Mechanismus für text-Widgets. Auch diese Verbesserungen sind aber *einzelner* Art.

Das Paket Wcb

Das Paket Wcb (**W**idget **c**all**b**ack, Widget-Rückruffunktion) bietet eine ganz andere, *allgemeine* Lösung für die genannten Probleme: Es definiert den Tcl-Befehl des Widgets neu. Die zentrale Prozedur `wcb::callback` lässt Sie beliebige Befehle mit bestimmten Operationen von Tk-entry-, Ttk-entry-, BWidget-Entry-, Tk-spinbox-, Ttk-spinbox-, Ttk-combobox-, listbox-, tablelist-, Ttk-treeview-, text- und ctext-Widgets verbinden. Diese Befehle werden automatisch im globalen Geltungsbereich aufgerufen, sobald die jeweilige Widget-Operation ausgeführt wird. Sie können verlangen, dass die Befehle vor oder nach der Widget-Operation laufen, Sie können also **before-** und **after-Rückruffunktionen** festlegen. Aus einer before-Rückruffunktion können Sie den Widget-Befehl mit der Prozedur `wcb::cancel` abbrechen oder seine Argumente mit `wcb::extend` oder `wcb::replace` ändern.

Neben diesen und vier weiteren allgemeinen Befehlen stellt das Paket Wcb vier Hilfsprozeduren für Tk-entry-, Ttk-entry-, BWidget-Entry-, Tk-spinbox-, Ttk-spinbox- und Ttk-combobox-Widgets bereit, außerdem einige before-`insert`-Rückruffunktionen für Tk-entry-, Ttk-entry-,

BWidget-Entry-, Tk-spinbox-, Ttk-spinbox- und Ttk-combobox-Widgets sowie für text- und ctext-Widgets. Sie können sie unverändert verwenden oder an Ihren Bedarf anpassen. Wie das geht, steht im Abschnitt Beispiele unten.

Das Paket Wcb ist in reinem Tcl/Tk geschrieben. Es ist dadurch vollständig plattformunabhängig und sehr einfach zu installieren. Es braucht Tcl und Tk in Version 8.4 oder höher.

Woher bekomme ich es?

Wcb steht auf dieser Webseite zum freien Herunterladen:

https://www.nemethi.de

Die Distributionsdatei heißt `wcb4.2.tar.gz` für UNIX und `wcb4_2.zip` für Windows. Beide Dateien enthalten dieselben Angaben, abgesehen von dem zusätzlichen Wagenrücklaufzeichen vor dem Zeilenvorschub am Ende jeder Zeile in den Textdateien für Windows.

Wcb ist auch in tklib enthalten, unter der Adresse

https://core.tcl.tk/tklib

Wie installiere ich es?

Installieren Sie das Paket als Unterverzeichnis eines der Verzeichnisse, die durch die Variable `auto_path` angegeben sind. Sie können es beispielsweise als Unterverzeichnis des Verzeichnisses `lib` innerhalb Ihrer Tcl/Tk-Installation installieren.

Um Wcb *unter UNIX* zu installieren, wechseln Sie mit `cd` in das gewünschte Verzeichnis und packen die Distributionsdatei `wcb4.2.tar.gz` aus:

```
gunzip -c wcb4.2.tar.gz | tar -xf -
```

Auf den meisten UNIX-Systemen kann dies durch Folgendes ersetzt werden:

```
tar -zxf wcb4.2.tar.gz
```

Beide Befehle legen ein Verzeichnis namens `wcb4.2` an, mit den Unterverzeichnissen `demos`, `doc` und `scripts`.

Unter Windows packen Sie die Distributionsdatei `wcb4_2.zip` mit WinZip oder einem anderen geeigneten Programm in das Verzeichnis `wcb4.2` aus, mit den Unterverzeichnissen `demos`, `doc` und `scripts`.

Beachten Sie, dass tklib das Wcb-Verzeichnis `demos` durch das Unterverzeichnis `wcb` des Verzeichnisses `examples` ersetzt. Bitte berücksichtigen Sie das, wenn Sie die Beispiele unten lesen.

Wie verwende ich es?

Damit Ihre Skripte auf die Befehle und Variablen des Pakets Wcb zugreifen können, müssen sie eine dieser Zeilen enthalten:

```
package require wcb ?version?
package require Wcb ?version?
```

Sie können jede der beiden Anweisungen verwenden, weil die Datei `wcb.tcl` beide Zeilen enthält:

```
package provide wcb ...
package provide Wcb ...
```

Sie können eine der beiden Zeilen aus `wcb.tcl` entfernen, wenn das Paket sich nicht unter zwei verschiedenen Namen bekannt machen soll. Damit beschränken Sie das Argument von `package require` natürlich auf einen einzigen Namen.

Weil das Paket Wcb in einem eigenen Namensraum namens `wcb` liegt, müssen Sie entweder den Befehl

```
namespace import wcb::pattern ?wcb::pattern ...?
```

aufrufen, um die benötigten *Prozeduren* zu importieren, oder qualifizierte Namen wie `wcb::callback` verwenden. In den Beispielen unten ist der zweite Weg gewählt.

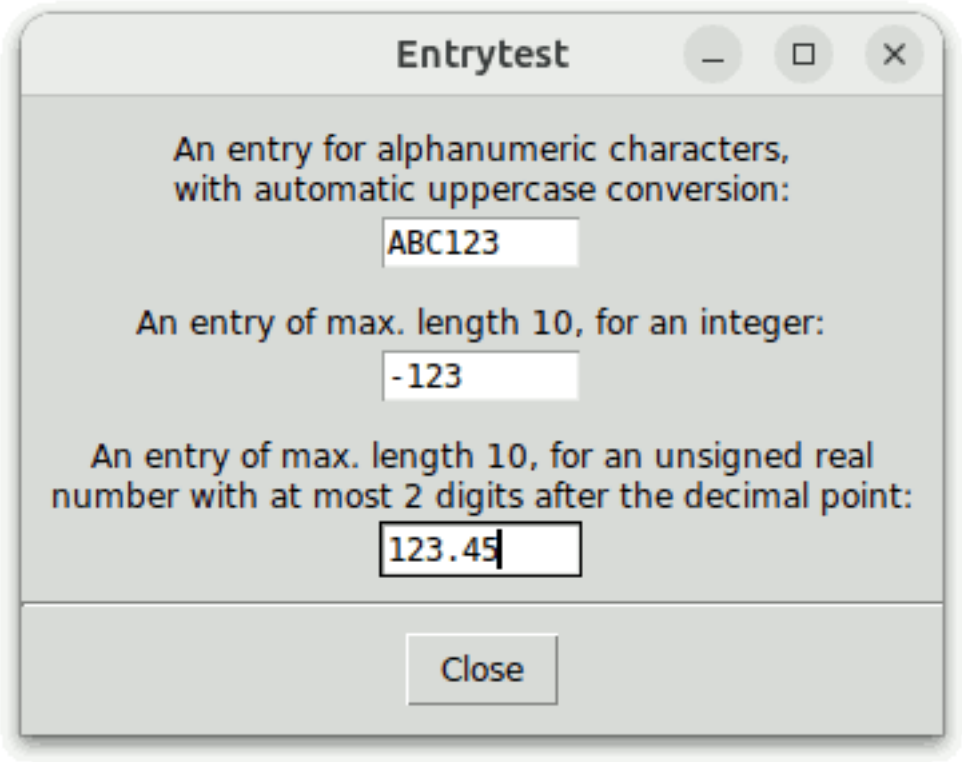
Auf Wcb-*Variablen* müssen Sie mit qualifizierten Namen zugreifen. Nur zwei Wcb-Variablen sind dafür gedacht, außerhalb des Namensraums `wcb` gelesen zu werden:

- Die Variable `wcb::version` enthält die aktuelle Versionsnummer des Pakets Wcb.
- Die Variable `wcb::library` enthält den Ort des Wcb-Installationsverzeichnisses.

Beispiele

Einige before-`insert`-Rückruffunktionen für entry-Widgets

Das Skript `entrytest.tcl` im Verzeichnis `demos` erzeugt drei entry-Widgets mit den Einschränkungen der folgenden Abbildung:



entrytest

Für das oberste entry-Widget `.e1` legen wir zwei before-`insert`-Rückruffunktionen aus dem Paket Wcb fest:

```
wcb::callback .e1 before insert wcb::checkStrForAlnum \
                                wcb::convStrToUpper
```

Damit das zweite entry-Widget `.e2` nur ganze Zahlen mit höchstens 10 Zeichen annimmt, verwenden wir wieder zwei before-`insert`-Rückruffunktionen aus Wcb:

```
wcb::callback .e2 before insert {wcb::checkEntryLen 10} \
                                 wcb::checkEntryForInt
```

Und schließlich die beiden Rückruffunktionen für das dritte entry-Widget `.e3`:

```
wcb::callback .e3 before insert {wcb::checkEntryLen 10} \
                                 checkNumber

#
# Callback procedure checkNumber
#
proc checkNumber {w idx str} {
    set newText [wcb::postInsertEntryText $w $idx $str]
    if {![regexp {^[0-9]*\.?[0-9]?[0-9]?$} $newText]} {
        wcb::cancel
    }
}
```

Dieses letzte Beispiel zeigt auch die Argumente der Rückruffunktionen, die mit dem Befehl `wcb::callback` festgelegt werden: Wird eine Rückruffunktion aufgerufen, werden der neue Name des ursprünglichen Tcl-Befehls des Widgets (also der Name des Proxy-Befehls) und die Argumente der jeweiligen Widget-Operation automatisch als Parameter angehängt. Weil wir `checkNumber` als before-Rückruffunktion für den Unterbefehl `insert` festgelegt haben, müssen seine letzten drei Argumente sein: der Name des Proxy-Widget-Befehls (`w`, hier `::_.e3`), der Index (`idx`) und die Zeichenkette (`str`), die unmittelbar vor dem Zeichen eingefügt werden soll, das der Index bezeichnet.

Beachten Sie, dass in der Argumentliste einer Wcb-Rückruffunktion dem Namen des Proxy-Widget-Befehls beliebig viele zusätzliche Argumente vorangestellt sein können. Die Prozedur `wcb::checkEntryLen` ist ein Beispiel für eine solche Rückruffunktion.

Der Befehl `wcb::postInsertEntryText`, den die Prozedur `checkNumber` aufruft, gibt den Text zurück, den das entry-Widget `w` enthielte, nachdem die Zeichenkette `str` vor dem Zeichen eingefügt wurde, das der Index `idx` bezeichnet. Ist dieser Text nicht der Anfang einer vorzeichenlosen reellen Zahl mit höchstens zwei Stellen nach dem Dezimalpunkt, rufen wir die Prozedur `wcb::cancel` auf, die den Befehl `insert` abbricht.

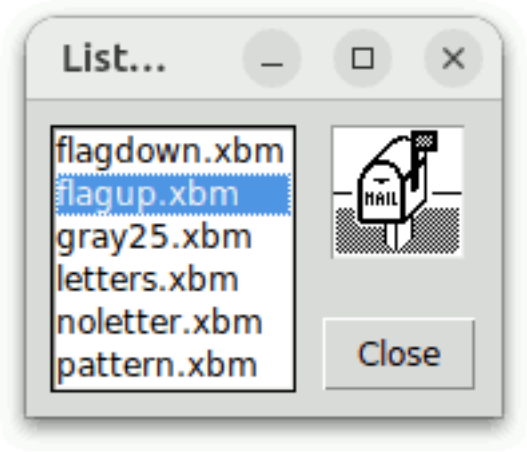
Ohne die Einschränkung, dass der Inhalt des dritten entry-Widgets nicht mit einem Vorzeichen beginnen darf, hätten wir statt `checkNumber` die Rückruffunktion `wcb::checkEntryForFixed` verwenden können:

```
wcb::callback .e3 before insert {wcb::checkEntryLen 10} \
                                {wcb::checkEntryForFixed 2}
```

Eine `selset`-Rückruffunktion für ein listbox-Widget

Bei einer listbox werden Sie wahrscheinlich am häufigsten eine Rückruffunktion für den Widget-Unterbefehl `selection set` festlegen wollen. Meist ist es egal, ob das eine before- oder eine after-Rückruffunktion ist. Der Befehl `wcb::callback` erwartet als Parameter die Kurzform `selset`. Entsprechend müssen Sie `selclear` übergeben, wenn Sie eine Rückruffunktion für die listbox-Operation `selection clear` festlegen.

Im folgenden Beispiel bauen wir eine listbox `.lb` mit den Namen der Bitmap-Dateien im Unterverzeichnis `images` des Wcb-Verzeichnisses `demos`. Das Verzeichnis `images` ist eine Kopie des gleichnamigen Unterverzeichnisses des Tk-8.6-Verzeichnisses `demos`. Wird ein Eintrag ausgewählt, zeigt die Rückruffunktion `showBitmap` die zugehörige Bitmap in der Beschriftung `.picture`.



listboxtest1

Hier der maßgebliche Ausschnitt aus dem Skript `listboxtest1.tcl` im Verzeichnis `demos`:

```
set dirName [file join [file dirname [info script]] images]

#
# Frame .spacer and listbox .lb
#
frame .spacer -width 7p
listbox .lb -height 0 -width 0 -background white
set pattern [file join $dirName *.xbm]
foreach pathName [lsort [glob $pattern]] {
    .lb insert end [file tail $pathName]
}

#
# Label .picture
#
label .picture -relief sunken -background white

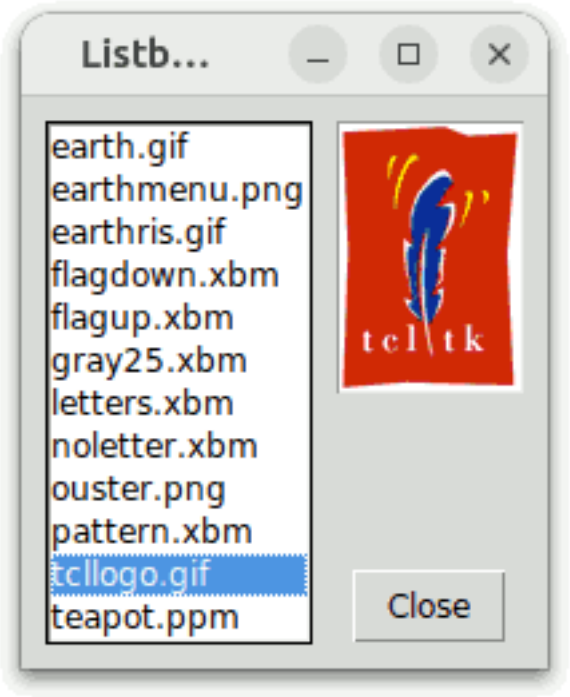
#
# Define a before-selset callback for .lb
#
wcb::callback .lb before selset showBitmap
```

```
#
# Callback procedure showBitmap
#
proc showBitmap {w first args} {
    global dirName
    set pathName [file join $dirName [$w get $first]]
    .picture configure -bitmap @$pathName
}
```

Zur Erinnerung: Die listbox-Operation `selection set` nimmt ein oder zwei Indizes als Argumente. Sie werden der Rückruffunktion automatisch als Parameter übergeben, zusammen mit dem neuen Namen des ursprünglichen Tcl-Befehls des listbox-Widgets, also dem Namen des Proxy-Befehls. Deshalb sind die Argumente der Rückruffunktion `showBitmap`: der Name des Proxy-Widget-Befehls (`w`, hier `::_.lb`), der erste Index (`first`) und das Schlüsselwort `args`, das für die leere Liste oder den optionalen zweiten Index steht, der an `selection set` übergeben wurde.

Eine `activate`-Rückruffunktion für ein listbox-Widget

Die listbox des vorigen Beispiels hat den Vorgabe-Auswahlmodus `browse`. Die before-`selset` -Rückruffunktion `showBitmap` läuft deshalb jedes Mal, wenn die Maus bei gedrückter Taste 1 von einem Element zu einem anderen gezogen wird. Was aber, wenn wir nicht nur die Bitmaps anzeigen wollen, sondern auch die Fotobilder im Unterverzeichnis `images` des Wcb-Verzeichnisses `demos`? Ein Fotobild zu laden ist deutlich aufwendiger als eine Bitmap zu laden. Einige Bilder lassen sich dann nicht schnell genug anzeigen, um der Maus zu folgen, wenn man mit ihr in der listbox blättert.



listboxtest2

Um das zu lösen, können wir den Auswahlmodus auf den selteneren Wert `single` setzen oder

dafür sorgen, dass die Bilder nicht beim Blättern mit der Maus angezeigt werden, sondern beim Loslassen der Taste 1. Der zweite Weg lässt sich mit einer `activate`-Rückruffunktion umsetzen, wie der folgende Ausschnitt aus dem Skript `listboxtest2.tcl` im Verzeichnis `demos` zeigt:

```
set dirName [file join [file dirname [info script]] images]
image create photo photoImage

#
# Frame .spacer and listbox .lb
#
frame .spacer -width 7p
listbox .lb -height 0 -width 0 -background white
set pattern [file join $dirName *]
foreach pathName [lsort [glob $pattern]] {
    .lb insert end [file tail $pathName]
}

#
# Label .picture
#
label .picture -relief sunken -background white

#
# Define a before-activate callback for .lb
#
wcb::callback .lb before activate showPicture

#
# Callback procedure showPicture
#
proc showPicture {w idx} {
    set leafName [$w get $idx]

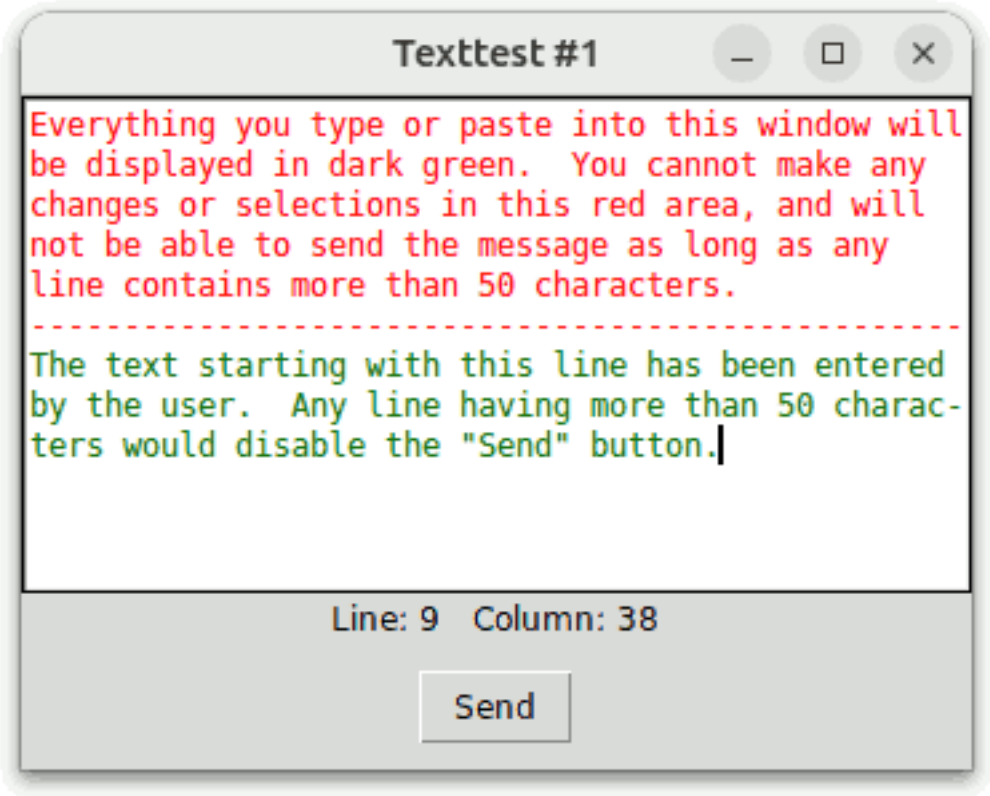
    #
    # When traversing the listbox with the arrow keys, the value
    # of idx can become -1 or the number of listbox elements,
    # hence the value of leafName can be an empty string:
    #
    if {$leafName eq ""} {
        return ""
    }

    global dirName
    set pathName [file join $dirName $leafName]
    if {[regexp {^\.(bmp|xbm)$} [file extension $pathName]]} {
        .picture configure -bitmap @$pathName -image ""
```

```
    } else {
        .picture configure -bitmap "" -image ""
        catch {
            photoImage configure -file $pathName
            .picture configure -bitmap "" -image photoImage
        }
    }
}
```

Sieben Rückruffunktionen für ein text-Widget

Das Skript `texttest1.tcl` im Verzeichnis `demos` erzeugt das text-Widget der folgenden Abbildung:



texttest1

Hier der maßgebliche Ausschnitt:

```
#
# Text .txt
#
set width 50
text .txt -width $width -height 12 -setgrid true -wrap none
    -background white \
          -font TkFixedFont
.txt tag configure prog -foreground red
.txt tag configure user -foreground DarkGreen
.txt insert end "Everything you type or paste into this window will\n"
```

```
     prog
.txt insert end "be displayed in dark green.  You cannot make any\n"
     prog
.txt insert end "changes or selections in this red area, and will\n"
     prog
.txt insert end "not be able to send the message as long as any\n"
     prog
.txt insert end "line contains more than $width characters.\n"
     prog
.txt insert end "--------------------------------------------------\n"
     prog
set limit [.txt index insert]

#
# Label .pos displaying the current cursor position
#
label .pos -textvariable pos

#
# Button .send (actually, it does not send anything)
#
button .send -text Send -command exit

#
# Define 5 before- and 2 after-callbacks for .txt
#
wcb::callback .txt before insert protectRedArea changeColor
wcb::callback .txt before delete protectRedArea
wcb::callback .txt before selset protectRedArea
wcb::callback .txt before motion displayPos
wcb::callback .txt  after insert "checkLines $width"
wcb::callback .txt  after delete "checkLines $width"

#
# Callback procedure protectRedArea
#
# The parameters following w can be interpreted either as
# "index string ?tagList string tagList ...?" (for an insert
# callback), or as "from ?to?" (for a delete callback),
# or as "from ?to from to ...?" (for a selset callback).
#
proc protectRedArea {w idx args} {
    global limit
    if {[$w compare $idx < $limit]} {
        wcb::cancel
    }
```

```
}

#
# Callback procedure changeColor
#
proc changeColor {w args} {
    wcb::extend user
}

#
# Callback procedure displayPos
#
proc displayPos {w idx} {
    set index [$w index $idx]
    scan $index "%d.%d" line column
    incr column

    global pos
    set pos [format "Line: %d   Column: %d" $line $column]
}

#
# Callback procedure checkLines
#
# The parameter args can be interpreted both as "index
# string ?tagList string tagList ...?" (for an insert
# callback) and as "from ?to?" (for a delete callback).
#
proc checkLines {maxCharsPerLine w args} {
    #
    # Display the new cursor position
    #
    displayPos $w insert

    #
    # Disable or enable the .send button
    #
    scan [$w index end] "%d" lastLine
    for {set line 1} {$line < $lastLine} {incr line} {
        scan [$w index $line.end] "%d.%d" dummy charsInLine
        if {$charsInLine > $maxCharsPerLine} {
            .send configure -state disabled
            return ""
        }
    }
    .send configure -state normal
```

```
}

. . .

displayPos .txt insert
focus .txt
```

Die Prozedur `protectRedArea` ist eine before-`insert`-, before-`delete`- und before-`selset`-Rückruffunktion. Sie prüft, ob die versuchte Änderung den rot dargestellten Textbereich beträfe. Ist das der Fall, ruft sie die Prozedur `wcb::cancel` auf, die den Befehl `insert`, `delete` bzw. `tag add sel` abbricht.

Die before-`insert`-Rückruffunktion `changeColor` ruft den Befehl `wcb::extend` auf, um das Tag `user` an die Argumentliste des Befehls `insert` anzuhängen. Die Vordergrundfarbe der vom Benutzer eingegebenen Zeichen wird dadurch `DarkGreen`.

Die Prozedur `displayPos` zeigt Zeile und Spalte zu dem Index, der ihr als zweites Argument übergeben wird. Dieser Index ist die Zielposition der Einfügemarke, wenn die Prozedur automatisch als before-`motion`-Rückruffunktion ausgelöst wird. Wie die unten besprochene Prozedur `checkLines` zeigt, wird sie auch nach einer `insert`- oder `delete`-Operation aufgerufen. Dann ist ihr zweites Argument die neue Position der Einfügemarke nach der Ausführung von `insert` oder `delete`. So lässt sich die Position der Einfügemarke vollständig verfolgen.

Es ist aufschlussreich, was geschieht, wenn wir `displayPos` als after- statt als before-`motion`-Rückruffunktion eintragen. In dem Fall müsste die Prozedur ihr zweites Argument ignorieren, und wir müssten die Zeile

```
set index [$w index $idx]
```

ersetzen durch

```
set index [$w index insert]
```

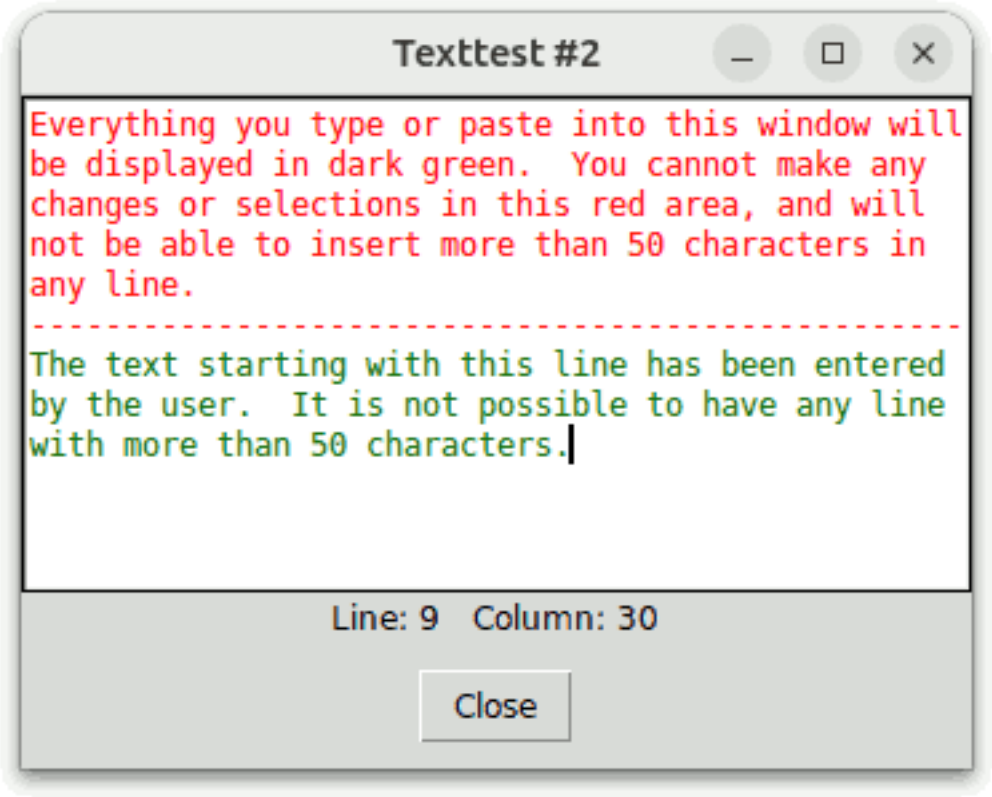
Der Grund: Der Wert des Arguments `idx`, das an `displayPos` übergeben wird, kann zum Beispiel `insert+1c` sein. Dabei meint `insert` die Position der Einfügemarke, *bevor* sie um ein Zeichen nach vorn rückt. Die after-`motion`-Rückruffunktion wird aber *nach* dem Verschieben der Einfügemarke ausgelöst, und dann zeigt die Marke `insert` bereits auf die neue Cursorposition. Deshalb reicht `[$w index $idx]` in einer after-`motion`-Rückruffunktion nicht, um die Position der Einfügemarke zu ermitteln.

Unsere letzte Prozedur `checkLines` ist sowohl eine after-`insert`- als auch eine after-`delete`-Rückruffunktion. Nach dem Aufruf von `displayPos`, der die neue Cursorposition anzeigt, schaltet sie die Schaltfläche `.send` aus oder ein, je nachdem, ob irgendeine Zeile des text-Widgets mehr als `$maxCharsPerLine` Zeichen enthält.

Beachten Sie, dass wir auch before-`replace`- und after-`replace`-Rückruffunktionen für den text-Widget-Unterbefehl `replace` hätten festlegen können, der in Tk 8.6 hinzukam. Das ist aber nicht nötig, weil die Vorgabebindungen des text-Widgets diesen Unterbefehl derzeit nicht verwenden.

Statt die Schaltfläche `.send` nur auszuschalten, wenn irgendeine Zeile mehr als `$maxCharsPerLine` Zeichen hat, können wir den Benutzer sogar daran hindern, längere Zeilen

einzugeben, als erlaubt sind. Das Skript `texttest2.tcl` im Verzeichnis `demos` zeigt, wie das gelingt, wenn Widget-Rückruffunktionen mit dem Rückgängig-Mechanismus für text-Widgets verbunden werden. Dieses Skript erzeugt das text-Widget der folgenden Abbildung:



texttest2

Der größte Teil des Codes in `texttest2.tcl` ist mit dem vorigen Beispiel identisch. Der wesentliche Unterschied ist eine neue Zeile, die den Rückgängig-Mechanismus für das text-Widget `.txt` einschaltet, und eine neue Fassung der Rückruffunktion `checkLines`:

```
. . .

.txt configure -undo yes

. . .

proc checkLines {maxCharsPerLine w args} {
    #
    # Undo the last insert or delete action if necessary
    #
    scan [$w index end] "%d" lastLine
    for {set line 1} {$line < $lastLine} {incr line} {
        scan [$w index $line.end] "%d.%d" dummy charsInLine
        if {$charsInLine > $maxCharsPerLine} {
            $w edit undo
            bell
            break
```

```
        }
    }

    #
    # Clear the undo and redo stacks, and display the new cursor
    position
    #
    $w edit reset
    displayPos $w insert
}

. . .
```

Diese Fassung der Rückruffunktion `checkLines` macht die letzte Bearbeitungsaktion rückgängig, wenn irgendeine Textzeile mehr Zeichen enthält als das erlaubte Maximum. Deshalb mussten wir den Aufruf der Prozedur `displayPos` ans Ende der Rückruffunktion verschieben, weil der Befehl `edit undo` die Position der Einfügemarke ändern kann.

Dieses Beispiel ließe sich auch ohne den Rückgängig-Mechanismus für text-Widgets umsetzen, indem man den zuletzt angenommenen Inhalt des Widgets zusammen mit der Cursorposition speichert und beides wiederherstellt, falls eine Zeile länger wird als erlaubt.

Stichwörter: wcb

wcbRef

Wcb-Befehlsreferenz

Für Wcb Version 4.2

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Der Befehl wcb::callback
- Der Befehl wcb::cbappend
- Der Befehl wcb::cbprepend
- Der Befehl wcb::cancel
- Der Befehl wcb::canceled
- Der Befehl wcb::extend
- Der Befehl wcb::replace
- Der Befehl wcb::pathname
- Der Befehl wcb::changeEntryText
- Der Befehl wcb::postInsertEntryLen
- Der Befehl wcb::postInsertEntryText
- Der Befehl wcb::postDeleteEntryText

- before-insert-Rückruffunktionen für entry-, spinbox- und combobox-Widgets
- before-insert-Rückruffunktionen für text- und ctext-Widgets

---

Der Befehl `wcb::callback`

NAME

`wcb::callback` – Widget-Rückruffunktionen abfragen, setzen und entfernen

ÜBERSICHT

```
wcb::callback widgetName before|after option ?callback callback ...?
```

BESCHREIBUNG

Fragt die Rückruffunktionen für das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox-, Ttk-combobox-, listbox-, tablelist-, Ttk-treeview-, text- oder ctext-Widget `widgetName`, das Argument `before` oder `after` und den Befehl zu `option` ab, setzt sie oder entfernt sie. Die Werte des Arguments `option` können sein:

- `insert`, `delete` oder `motion`, für ein Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox-, Ttk-combobox-, text- oder ctext-Widget;
- `replace`, für ein text- oder ctext-Widget;
- `activate`, für ein listbox-, tablelist- oder Ttk-treeview-Widget;
- `selset` oder `selclear`, für ein listbox-, tablelist-, Ttk-treeview-, text- oder ctext-Widget;
- `seladd` oder `seltoggle`, für ein Ttk-treeview-Widget;
- `cellselset` oder `cellselclear`, für ein tablelist- oder Ttk-treeview-Widget;
- `cellseladd` oder `cellseltoggle`, für ein Ttk-treeview-Widget;
- `activatecell`, für ein tablelist-Widget.

Ist nach dem Parameter `option` kein Argument angegeben, gibt die Prozedur nur die aktuelle before- bzw. after-Rückruffunktionsliste für die gegebene Widget-Operation zurück.

Andernfalls:

- Ist mindestens eines der Argumente nach dem Parameter `option` eine nichtleere Zeichenkette, dann:
- Sind alle Argumente nach dem Parameter `option` leer, hebt die Prozedur die Eintragung aller zugehörigen before- oder after-Rückruffunktionen für das gegebene Widget auf und gibt eine leere Zeichenkette zurück.
  - wird der Befehl für dieses Widget zum ersten Mal mit mindestens einem nichtleeren Argument nach dem Parameter `option` aufgerufen, benennt die Prozedur den Tcl-Befehl `::widgetName` in `::_widgetName` um und baut eine neue Prozedur `::widgetName`, in der die Ausführung der Widget-Operationen zu den oben genannten Werten von `option` von Aufrufen der zugehörigen before-Rückruffunktionen eingeleitet und von Aufrufen der zugehörigen after-Rückruffunktionen im globalen Geltungsbereich gefolgt wird;
  - sie setzt die Rückruffunktionsliste auf die aus diesen Argumenten gebildete Liste und gibt die neue Liste zurück.

Wird eine Rückruffunktion für das Widget `widgetName` aufgerufen, werden der Name des **Proxy-Befehls** `::_widgetName` und die Befehlsargumente automatisch als Parameter angehängt. Mit dem Befehl `wcb::pathname` holen Sie den ursprünglichen

Widget-Pfadnamen.

Die folgende Tabelle zeigt die Widget-Unterbefehle zu den oben genannten Werten von `option`, zusammen mit den Argumenten dieser Unterbefehle:

Widget	**option**	**Unterbefehl**	**Argumente**
Tk entry, ttk::entry, BWidget Entry, Tk spinbox, ttk::spinbox, oder ttk::combobox	insert	insert	index string
delete	delete	from ?to?	
motion	icursor	index	
listbox	activate	activate	index
selset	selection set	first ?last?	
selclear	selection clear	first ?last?	
tablelist::tablelist	activate	activate	index
selset	selection set	first ?last? (indexList oder firstIndex lastIndex)	
selclear	selection clear	first ?last? (indexList oder firstIndex lastIndex)	
activatecell	activatecell	cellIndex	
cellselset	cellselection set	first ?last? (cellIndexList oder firstCell lastCell)	
cellselclear	cellselection clear	first ?last? (cellIndexList oder firstCell lastCell)	
ttk::treeview	activate	focus	item
selset	selection set	itemList	
seladd	selection add	itemList	
selclear	selection remove	itemList	
seltoggle	selection toggle	itemList	
cellselset	cellselection set	first ?last? (cellList oder firstCell lastCell)	

Widget	**option**	**Unterbefehl**	**Argumente**
cellseladd	cellselection add	first ?last? (cellList oder firstCell lastCell)	
cellselclear	cellselection remove	first ?last? (cellList oder firstCell lastCell)	
cellseltoggle	cellselection toggle	first ?last? (cellList oder firstCell lastCell)	
text oder ctext	insert	insert	index string ?tagList string tagList ...?
delete	delete	from ?to?	
replace	replace	from to string ?tagList string tagList ...?	
motion	mark set insert	index	
selset	tag add sel	from ?to from to ...?	
selclear	tag remove sel	from ?to from to ...?	

HINWEISE:

1. Die Wörter `before`, `after`, `insert`, `delete`, `replace`, `motion` und `activate` (letzteres nicht bei tablelist-Widgets) dürfen Sie auf mindestens ein Zeichen kürzen. Ebenso genügen die ersten vier Zeichen der Wörter `selset`, `selclear`, `seladd` und `seltoggle`, die ersten acht Zeichen von `cellselset`, `cellselclear`, `cellseladd` und `cellseltoggle` und die ersten neun Zeichen von `activatecell`, damit Wcb diese Optionen erkennt.
2. Nach einem erfolgreichen Aufruf dieses Befehls mit mindestens einer nichtleeren Rückruffunktion nach dem Argument `option` können Sie entweder die neue Prozedur `widgetName` oder den Proxy-Befehl `::_widgetName` verwenden, um jede gültige Operation auf dem Widget `widgetName` auszuführen. Verwenden Sie den Proxy-Befehl `::_widgetName`, wenn die Rückruffunktionen beim Ausführen des jeweiligen Widget-Unterbefehls nicht aufgerufen werden sollen.
3. Wird ein Widget `widgetName` zerstört, für das `wcb::callback` den zugehörigen Tcl-Befehl durch eine neue Prozedur ersetzt hat, löscht der Tcl-Interpreter den Proxy-Befehl `::_widgetName` automatisch. Das gilt nicht für ein BWidget Entry, ein tablelist- oder ein ctext-Widget. Die neue Widget-Prozedur `widgetName` bliebe bestehen, Wcb sorgt aber dafür, dass sie aus einem Aufräumskript gelöscht wird, das an das Ereignis `<Destroy>` gebunden ist. Dieses Aufräumskript gehört zu einem Bindungs-Tag namens `WcbCleanup`, das der Liste der Bindungs-Tags des Widgets angehängt wird, wenn zum ersten Mal Rückruffunktionen dafür eingetragen werden. Bei einem BWidget Entry, einem tablelist- oder einem ctext-Widget löscht dieses Skript auch den Proxy-Befehl `::_widgetName`.
4. Das genannte Aufräumskript hebt außerdem die Eintragung aller für `widgetName` festgelegten Rückruffunktionen auf. Ein später mit demselben Pfad erzeugtes Widget erbt sie dadurch nicht von dem gerade gelöschten Widget. Das kann in manchen Anwendungen

wichtig sein. Entfernen Sie `WcbCleanup` deshalb nicht aus der Liste der Bindungs-Tags des gegebenen Widgets.

SCHLÜSSELWÖRTER

callback, widget, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, listbox, tablelist, Ttk treeview, text, ctext

---

Der Befehl `wcb::cbappend`

NAME

`wcb::cbappend` – An eine Rückruffunktionsliste anhängen

ÜBERSICHT

```
wcb::cbappend widgetName before|after option ?callback callback ...?
```

BESCHREIBUNG

Dieser Befehl ist nahezu identisch mit `wcb::callback`. Der einzige Unterschied: `wcb::cbappend` *hängt* die nach dem Parameter `option` angegebenen Argumente an die aktuelle Rückruffunktionsliste an, falls eine vorhanden ist, während `wcb::callback` die alten Rückruffunktionen durch diese Argumente *ersetzt*.

SCHLÜSSELWÖRTER

callback, append, widget

Der Befehl `wcb::cbprepend`

NAME

`wcb::cbprepend` – Einer Rückruffunktionsliste voranstellen

ÜBERSICHT

```
wcb::cbprepend widgetName before|after option ?callback callback ...?
```

BESCHREIBUNG

Dieser Befehl ist nahezu identisch mit `wcb::callback`. Der einzige Unterschied: `wcb::cbprepend` *stellt* die nach dem Parameter `option` angegebenen Argumente der aktuellen Rückruffunktionsliste *voran*, falls eine vorhanden ist, während `wcb::callback` die alten Rückruffunktionen durch diese Argumente *ersetzt*.

SCHLÜSSELWÖRTER

callback, prepend, widget

Der Befehl `wcb::cancel`

NAME

`wcb::cancel` – Einen Widget-Befehl abbrechen

ÜBERSICHT

```
wcb::cancel ?script?
```

BESCHREIBUNG

Diese Prozedur ist dafür gedacht, aus einer before-Rückruffunktion für einen Widget-Befehl

aufgerufen zu werden. Sie bricht die Ausführung dieses Befehls und der übrigen Rückruffunktionen ab und wertet das Argument `script` im globalen Geltungsbereich aus. Ist dieses Argument nicht angegeben, ist die Vorgabe der Befehl `bell`.

Der Rückgabewert ist der von `script`, wenn dieses Argument angegeben und nichtleer ist. Sonst gibt der Befehl eine leere Zeichenkette zurück.

SCHLÜSSELWÖRTER

cancel, command, widget

Der Befehl `wcb::canceled`

NAME

`wcb::canceled` – Den Abbruchzustand eines Widget-Befehls abfragen

ÜBERSICHT

```
wcb::canceled widgetName option
```

BESCHREIBUNG

Gibt den Wert `1` zurück, wenn der jüngste Aufruf der Widget-Operation zu `widgetName` und `option` von einer before-Rückruffunktion durch den Aufruf von `wcb::cancel` abgebrochen wurde; sonst ist der Rückgabewert `0`. Die Argumente müssen dieselben Einschränkungen erfüllen wie beim Befehl `wcb::callback`.

SCHLÜSSELWÖRTER

cancel, command, widget

Der Befehl `wcb::extend`

NAME

`wcb::extend` – Die Argumentliste eines Widget-Befehls erweitern

ÜBERSICHT

```
wcb::extend ?arg arg ...?
```

BESCHREIBUNG

Diese Prozedur ist dafür gedacht, aus einer before-Rückruffunktion für einen Widget-Befehl aufgerufen zu werden. Sie hängt die in den optionalen Parametern `arg` gegebenen Werte an die Argumentliste dieses Befehls an. Die neue Argumentliste wird auch an die übrigen Rückruffunktionen für diesen Befehl weitergegeben.

Diese Prozedur gibt ihre Parameter einfach an den Befehl `lappend` weiter, aufgerufen für die Argumentliste der jeweiligen Widget-Operation.

SCHLÜSSELWÖRTER

extend, argument, command, widget

Der Befehl `wcb::replace`

NAME

wcb::replace – Argumente eines Widget-Befehls durch neue ersetzen

ÜBERSICHT

```
wcb::replace first last ?arg arg ...?
```

BESCHREIBUNG

Diese Prozedur ist dafür gedacht, aus einer before-Rückruffunktion für einen Widget-Befehl aufgerufen zu werden. Sie ersetzt die Argumente mit den Indizes `first` bis `last` dieses Befehls durch die optionalen Parameter `arg`. Die neue Argumentliste wird auch an die übrigen Rückruffunktionen für diesen Befehl weitergegeben. Die Argumente sind ab `0` nummeriert (siehe die Tabelle in der Beschreibung des Befehls `wcb::callback`).

Diese Prozedur gibt ihre Parameter einfach an den Befehl `lreplace` weiter, aufgerufen für die Argumentliste der jeweiligen Widget-Operation.

SCHLÜSSELWÖRTER

replace, argument, command, widget

Der Befehl `wcb::pathname`

NAME

wcb::pathname – Den Pfadnamen des Widgets zu einem Tcl-Befehlsnamen abfragen

ÜBERSICHT

```
wcb::pathname proxyCmd
```

BESCHREIBUNG

Diese Prozedur gibt den Pfadnamen des Widgets zurück, das zu dem Tcl-Befehlsnamen proxyCmd gehört, der an eine Rückruffunktion übergeben wurde.

Wird eine before- oder after-Rückruffunktion für ein Widget aufgerufen, wird der Name des zugehörigen Proxy-Befehls automatisch als Parameter angehängt. Innerhalb einer Rückruffunktion holt diese Prozedur den Pfadnamen des Widgets aus dem Befehlsnamen, der der Rückruffunktion als Argument übergeben wurde. Sie gibt einfach den Zeichenkettenbereich zurück, der aus `proxyCmd` durch Entfernen des Präfixes `"::_"` entsteht.

SCHLÜSSELWÖRTER

path name, command, widget

Der Befehl `wcb::changeEntryText`

NAME

wcb::changeEntryText – Den Text eines Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widgets ändern

ÜBERSICHT

```
wcb::changeEntryText widgetName string
```

BESCHREIBUNG

Ersetzt den Text des Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widgets `widgetName` durch `string`, über die Operationen `delete` und `insert`. Bricht eine before-`delete`-Rückruffunktion den ersten Unterbefehl ab, kehrt die Prozedur zurück, ohne den neuen Text einzufügen. Bricht eine before-`insert`-Rückruffunktion die zweite Operation ab, stellt der Befehl den ursprünglichen Inhalt des Widgets wieder her.

Die Prozedur behält die Position der Einfügemarke bei. Der Rückgabewert ist `1` bei Erfolg und `0` bei Fehlschlag, also wenn eine der versuchten Operationen von einer before-Rückruffunktion abgebrochen wurde.

SCHLÜSSELWÖRTER

Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

Der Befehl `wcb::postInsertEntryLen`

NAME

`wcb::postInsertEntryLen` – Die Länge abfragen, die der Text in einem Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget nach dem Einfügen hätte

ÜBERSICHT

```
wcb::postInsertEntryLen widgetName string
```

BESCHREIBUNG

Gibt die Länge des Textes zurück, den das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `widgetName` nach dem Einfügen von `string` enthielte.

SCHLÜSSELWÖRTER

insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

Der Befehl `wcb::postInsertEntryText`

NAME

`wcb::postInsertEntryText` – Den Text abfragen, den ein Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget nach dem Einfügen hätte

ÜBERSICHT

```
wcb::postInsertEntryText widgetName index string
```

BESCHREIBUNG

Gibt den Text zurück, den das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `widgetName` enthielte, nachdem `string` vor dem Zeichen eingefügt wurde, das `index` bezeichnet.

SCHLÜSSELWÖRTER

insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

Der Befehl `wcb::postDeleteEntryText`

NAME

`wcb::postDeleteEntryText` – Den Text abfragen, den ein Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget nach dem Löschen hätte

ÜBERSICHT

```
wcb::postDeleteEntryText widgetName from ?to?
```

BESCHREIBUNG

Gibt den Text zurück, den das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `widgetName` enthielte, nachdem die Zeichen ab dem von `from` bezeichneten und unmittelbar vor `to` gelöscht wurden. Ist `to` nicht angegeben, ist der Rückgabewert der Text, den das Widget nach dem Löschen des einzelnen von `from` gegebenen Zeichens enthielte.

HINWEIS: Dieser Befehl hat eine veränderliche Anzahl von Argumenten (2 oder 3), weil der Unterbefehl `delete` des zu `widgetName` gehörenden Tcl-Befehls entweder einen oder zwei Indizes als Argumente erwartet. Der richtige Aufruf aus einer before-`delete`-Rückruffunktion sieht deshalb so aus wie im folgenden Beispiel:

```
proc myBeforeDeleteCallback {w args} {
    #
    # Get the text that would be contained in the widget after
    # deleting the characters specified by $args, which stands
    # for the one or two arguments passed to delete; pass these
    # arguments to wcb::postDeleteEntryText by expanding $args
    #
    set newText [eval [list wcb::postDeleteEntryText $w] $args]
    if {!some_condition_on_$newText} {
        wcb::cancel
    }
}
```

Die folgende alternative, elegantere Lösung braucht Tcl/Tk 8.5 oder neuer:

```
proc myBeforeDeleteCallback {w args} {
    # . . .
    set newText [wcb::postDeleteEntryText $w {*}$args]
    . . .
}
```

SCHLÜSSELWÖRTER

delete, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

before-`insert`**-Rückruffunktionen für entry-, spinbox- und combobox-Widgets**

NAME

`wcb::checkStrFor*`, `wcb::convStrTo*`, `wcb::checkEntryFor*`,

wcb::checkEntryLen – before-insert-Rückruffunktionen für Tk-entry-, Ttk-entry-, BWidget-Entry-, Tk-spinbox-, Ttk-spinbox- und Ttk-combobox-Widgets

ÜBERSICHT

```
wcb::checkStrForRegExp  exp w idx str

wcb::checkStrForAlpha       w idx str
wcb::checkStrForNum         w idx str
wcb::checkStrForAlnum       w idx str

wcb::convStrToUpper         w idx str
wcb::convStrToLower         w idx str

wcb::checkEntryForInt       w idx str
wcb::checkEntryForUInt  max w idx str
wcb::checkEntryForReal      w idx str
wcb::checkEntryForFixed cnt w idx str

wcb::checkEntryLen      len w idx str
```

BESCHREIBUNG

Die Rückruffunktion `wcb::checkStrForRegExp` prüft, ob der reguläre Ausdruck `exp` die Zeichenkette `str` trifft, die in das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `w` eingefügt werden soll. Trifft er sie nicht, bricht sie die Operation `insert` ab.

Die drei anderen Rückruffunktionen `wcb::checkStrFor*` prüfen, ob die Zeichenkette `str`, die in das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `w` eingefügt werden soll, alphabetisch, numerisch bzw. alphanumerisch ist. Ist sie es nicht, brechen sie die Operation `insert` ab. Diese Prozeduren rufen nur die Rückruffunktion `wcb::checkStrForRegExp` auf und übergeben ihr die Unicode-Muster `{^[[:alpha:]]*$}`, `{^[[:digit:]]*$}` und `{^[[:alnum:]]*$}`.

Die Rückruffunktionen `wcb::convStrTo*` ersetzen die Zeichenkette `str`, die in das Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `w` eingefügt werden soll, durch ihre Entsprechung in Groß- bzw. Kleinbuchstaben.

Die Rückruffunktionen `wcb::checkEntryFor*` prüfen, ob der Text im Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `w` nach dem Einfügen der Zeichenkette `str` vor dem Zeichen, das der Index `idx` bezeichnet, der Anfang einer ganzen Zahl, einer vorzeichenlosen ganzen Zahl nicht größer als `max`, einer reellen Zahl bzw. einer reellen Zahl im Festkommaformat mit höchstens `cnt` Stellen nach dem Dezimalpunkt wäre. Ist das nicht der Fall, brechen sie die Operation `insert` ab. `max` und `cnt` sollen nichtnegative Zahlen oder `*` sein. `max = *` bedeutet: keine obere Schranke für den Wert des Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widgets, während `cnt = *` für eine unbegrenzte Anzahl von Stellen nach dem Dezimalpunkt steht.

Die Rückruffunktion `wcb::checkEntryLen` prüft, ob die Länge des Textes im Tk- oder Ttk-entry-, BWidget-Entry-, Tk- oder Ttk-spinbox- oder Ttk-combobox-Widget `w` nach dem Einfügen der Zeichenkette `str` größer als `len` wäre. Ist das der Fall, bricht sie die Operation `insert` ab.

Diese Rückruffunktionen sind in der Datei `wcbEntry.tcl` im Verzeichnis `scripts` implementiert. Sie geben eine leere Zeichenkette zurück.

SCHLÜSSELWÖRTER

callback, insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

before-`insert`-Rückruffunktionen für text- und ctext-Widgets

NAME

`wcb::checkStrsFor*`, `wcb::convStrsTo*` – before-`insert`-Rückruffunktionen für text- und ctext-Widgets

ÜBERSICHT

```
wcb::checkStrsForRegExp exp w idx args

wcb::checkStrsForAlpha      w idx args
wcb::checkStrsForNum        w idx args
wcb::checkStrsForAlnum      w idx args

wcb::convStrsToUpper        w idx args
wcb::convStrsToLower        w idx args
```

BESCHREIBUNG

Die Rückruffunktion `wcb::checkStrsForRegExp` prüft, ob der reguläre Ausdruck `exp` die Zeichenketten trifft, die in das text- oder ctext-Widget `w` eingefügt werden sollen und in der Liste `args` der Form `string ?tagList string tagList ...?` stehen. Trifft er sie nicht, bricht sie die Operation `insert` ab.

Die drei anderen Rückruffunktionen `wcb::checkStrsFor*` prüfen, ob die Zeichenketten, die in das text- oder ctext-Widget `w` eingefügt werden sollen und in der Liste `args` der Form `string ?tagList string tagList ...?` stehen, alphabetisch, numerisch bzw. alphanumerisch sind. Sind sie es nicht, brechen sie die Operation `insert` ab. Diese Prozeduren rufen nur die Rückruffunktion `wcb::checkStrsForRegExp` auf und übergeben ihr die Unicode-Muster `{^[[:alpha:]\n]*$}`, `{^[[:digit:]\n]*$}` und `{^[[:alnum:]\n]*$}`.

Die Rückruffunktionen `wcb::convStrsTo*` ersetzen die Zeichenketten, die in das text- oder ctext-Widget `w` eingefügt werden sollen und in der Liste `args` der Form `string ?tagList string tagList ...?` stehen, durch ihre Entsprechungen in Groß- bzw. Kleinbuchstaben.

Diese Rückruffunktionen sind in der Datei `wcbText.tcl` im Verzeichnis `scripts` implementiert. Sie geben eine leere Zeichenkette zurück.

SCHLÜSSELWÖRTER

callback, insert, text, ctext, widget

Stichwörter: wcbRef

Lizenz

Deutsche Übersetzung © 2026 Gregor Ebbing.

Die Seiten wurden aus den Originalen nach Markdown umgewandelt und ins Deutsche übersetzt; Auszeichnung ohne Platz im Format (Verweise ohne Ziel, Layoutbilder) kann dabei wegfallen. Das sind die Änderungen; für Teile unter der Mozilla Public License 1.1 (tDOM) ist es die Beschreibung der Änderungen nach Abschnitt 3.3.

Die Übersetzung steht unter denselben Bedingungen wie das jeweilige englische Original. Diese Bedingungen folgen. Die Copyright-Angaben der Originalseiten stehen bei jeder Seite.

Widget callback package Wcb, version 4.2 Copyright (c) 1999-2025 Csaba Nemethi (E-mail: csaba.nemethi@t-online.de)

This library is free software; you can use, modify, and redistribute it for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions.

This software is distributed WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Stichwortverzeichnis

I

index 3

W

wcb 16

wcbRef 27